

탁찬홍 Backend Developer

Introduction

장애 시나리오를 설계 단계부터 상정하고, 문제를 수치로 재현해 해결하는 백엔드 개발자입니다.

대규모 트래픽 환경에서 분산 락·비동기 이벤트 아키텍처로 데이터 정합성을 확보하고, Hybrid Search 랭킹 역전 버그를 발견해 관련 파트너 포함율을 0.67 → 6.3 으로 개선한 경험이 있습니다.

Spring AI-MCP 기반 서버 진단 에이전트로 카카오 PlayMCP 공식 승인을 받았으며, Apache/Amoro 오픈소스 버그 수정에도 기여했습니다.

현재 B2B 블록체인 결제 플랫폼 개발 리드로 참여하며 도메인을 확장하고 있습니다.

Open Source

Apache/Amoro [\[PR 링크\]](#)

기간 : 2026.03 ~ 2026.04

- Mixed-Hive 비파티션 테이블 Iceberg, Hive 테스트 커버리지 추가(TestRewriteFiles / TestOverwriteFiles)
- Iceberg commit 과 Hive location 불일치로 인한 데이터 누락 버그를 발견·수정
(UpdateHiveFiles 클래스에 location consistency 검증 로직 추가)

Skills

[Backend] **Java / Spring Boot / Spring Data JPA / QueryDSL / TypeScript / NestJS**

[Database] **MySQL / Redis / MongoDB**

[Infra] **Docker / AWS EC2·RDS·S3 / GitHub Actions**

Project

[무역 파트너 매칭 및 XRPL 에스스로 결제 B2B 플랫폼]

[\[GITHUB\]](#)[\[배포링크\]](#)

일정 : 2026.04 ~ 진행 중

기술 스택 : TypeScript, NestJS, MongoDB Atlas, Redis, XRPL, Docker, Railway, GitHub Actions, Claude Code

참여인원 : 4 명 (대표(기획), BE 1, FE 1, UI/UX 1) / 팀 리드로 API 스펙 및 기술 방향 협의, 파트너사 기술 미팅 참여

서비스 : 무역거래 바이어·셀러 검색 및 매칭 후 XRPL / RLUSD 무역대금 결제 플랫폼

• [XRPL 에스스로 이벤트 유실 방지 – 원자적 발행 보장]

문제: 블록체인 트랜잭션 특성상 네트워크 지연·서버 다운타임 발생 시 EscrowCreate 이벤트 유실 위험

해결: Outbox 패턴 + MongoDB Change Streams 로 Outbox 이벤트 기록을 결제 상태 변경과 동일 트랜잭션으로 묶어 이벤트 유실을 구조적으로 차단

결과: 서버 재시작·네트워크 단절 시나리오 E2E 테스트로 이벤트 누락 0 건 검증

• [MQ 장애·XRPL 제출 실패 복구 및 상태 동기화 보장]

문제: MQ 장애 또는 XRPL 네트워크 오류 시 제출 성공/실패 상태가 DB 와 불일치할 위험

해결: Bull Queue(AOF) + 지수 Backoff(10 회, 초기 5s) 재시도로 MQ 장애 대응, 제출 성공/실패/타임아웃 시나리오별 Recovery Scheduler 로 DB-XRPL 상태 동기화

결과: Mock 테스트로 DB-XRPL 상태 불일치 0 건 검증, XRPL Testnet 에서 초기금·중도금·잔금 3 단계 에스스로 결제 흐름 완성

- **[Hybrid Search 도입 및 반복 개선으로 파트너 매칭 검색 품질 향상]**

문제: 1536 차원 기반 vector search 만 운영 시 도메인 특화 키워드가 유사도 계산에 묻혀 **관련 파트너 누락**

→ 3 개 쿼리 기준 상위 10 개 결과 내 관련 파트너 평균 0.67/10

해결: vector-keyword 병렬 검색 도입 → 가중합의 점수 스케일 불일치로 랭킹 역전 발견 → RRF 로 전환, ghost rank 패널티 고정으로 오탐 개선

결과: 동일 3 개 쿼리 기준 **관련 파트너 포함율 평균 0.67/10 → 6.3/10 개선**, 도메인 무관 노이즈 제거, 현재 파트너사 대상 PoC 진행중

['NAFAL' 경매 플랫폼 – 해커톤 최우수상 및 개인 리팩토링]

[\[GITHUB\]](#)[\[BLOG\]](#)

일정 : 2025.11 ~ 2026.03 (300 만건 대규모 트래픽을 가정한 시스템 고도화 및 리팩토링) / 2025.08 (해커톤 참여 및 최우수상)

기술 스택 : JAVA 17, Spring boot (3.4.5), MySQL(8.0), JPA, QueryDSL, Redis, AWS, Github Action

참여인원 : 해커톤 (6 인: BE 2, UX/UI 1, PM 3) / 리팩토링 (개인)

서비스 : 기업 폐기 예정 자산 재판매 플랫폼

테스트 환경 : K6 부하테스트 / DB (t2.medium), App (t2.medium), Redis (t2.small) / 서비스 급성장 (데이터 300 만건 이상) 가정

- **[경매 마감 골든타임의 입찰 유실 방지 및 서비스 전반 성능 저하 개선]**

문제: 경매 마감 임박 트래픽 집중 시 비관 락이 트랜잭션 종료까지 DB Connection 장기 점유 및 **입찰 요청 5% 500 에러(TCP reset) 발생**

해결: 낙관 락 + 재시도는 경합 집중 시 $O(N^2)$ 폭발 위험으로 제외 → **Redisson 분산 락**으로 락을 Redis 로 분리

결과: (K6 ramping-vus 입찰/조회 각각 최대 200VU 부하) **조회 응답 1,321ms → 70ms(94.7%), 입찰 응답 1,649ms -> 665ms(59.7%)**

- **[무손실 비동기 처리를 통한 유저 등급 데이터 정합성 보장]**

문제: 입찰·유저 트랜잭션 간 **cross lock Deadlock 발생**, 비동기 처리 시 부분 실패로 유저 등급 산정에 직결되는 **참여도 데이터 유실 발생**

해결: TransactionalEventListener 로 트랜잭션 경계 분리, Claim Check 패턴으로 Event Store 에 이벤트 저장 및 Consumer Log 로 실행하는 아키텍처 구축

결과: **Deadlock 제거(100 개 스레드 동시성 테스트 통과)**, Mockito 테스트로 부분 실패 후 복구 검증을 통해 참여도 데이터 정합성 보장

- **[인프라 증설 없이 쿼리 튜닝으로 대규모 데이터 탐색 지연 99% 개선]**

문제: 300 만 건 환경 부하테스트 결과 인덱스 부재 및 Count 쿼리 병목으로 **경매 조회 요청 Timeout (p95 30s) 발생**

해결: Covering Index & Deferred Join 적용, page -> slice 전환, Page Count 쿼리 Redis Caching 으로 집계 비용 제거

결과: (K6 ramping-vus 최대 100VU 부하) **조회 응답 30s -> 62ms (99.2% 개선)**

[Spring 서버 진단 MCP 'Server Doctor' – 카카오 PlayMCP 공식 승인/배포]

[\[GITHUB\]](#) [\[BLOG\]](#) [\[배포링크\]](#)

일정 : 2026.01 ~ 2026.03

기술 스택 : JAVA 21, Spring boot (3.4.2), Spring AI, Python, MySQL(8.0), JPA, AWS, Github Action

참여인원 : 개인

서비스 : Spring 서버의 Log/Metrics 를 로그수집기(Forwarder)가 추출하여 LLM 을 통해 자동 진단하는 MCP

테스트 환경 : K6 부하테스트 / 로컬 환경 (DB-App 동일 머신)

- **[Sidecar 기반 Push 아키텍처로 사용자 서버 보안성 확보]**

문제: Log/Metrics 수집을 위해 외부 포트(9090) 개방 운영 중 로그 유출 및 개인정보 보안 위험 인지

해결: 사용자 서버에 9090 포트로 가져오는 Pull 방식은 보안 위험으로 제외 -> **Forwarder 를 Sidecar 형태로 내부 배치**해 Push 방식으로 전송

결과: 외부 포트 노출 없이 기존 서버와 독립적인 로그 수집 구조 확보

- [로그 수집 파이프라인 DB 병목 해소 및 TPS 68.9% 향상]

문제: 수집 서버 장애 복구 시 Queue 에 누적된 로그가 한번에 유입되며, 단건 Insert 네트워크 병목 및 중복 키 **S-Lock → X-Lock** 승격으로 **Deadlock 발생**

해결: 단건 Insert -> JDBC Bulk Insert 로 처리량 향상 및 @Retryable 로 간헐적 Deadlock 자동복구

결과: **TPS 68.9% 향상**, 응답시간 65.3% 개선, Deadlock 서비스 장애 0 건

- [시스템 연쇄 장애 방어를 위한 유량제어 및 생존성 확보]

문제: 통신 장애 시 **무한 재시도로 OOM 이 발생**하여 사용자 서버 다운되는 위험 및 **로그 유실** 가능성 발견

해결: Queue+ Jitter + Timeout 1 분으로 연쇄 장애 방어 및 SHA-1 멍등성 키 + ON DUPLICATE KEY UPDATE 로 중복 제거

결과: 장애 상황에도 **Forwarder 의 가용성 유지**, **SHA-1 멍등성 키로 중복 삽입 0 건 보장**

Resume

경북대학교 글로벌소프트웨어융합전공(복수전공) / 건설방재공학과(본전공)

기간 : 2021.03 ~ 현재 (2027 년 2 월 졸업예정)

SQLD

취득일 : 2025.12

Personal Information

GITHUB : <https://github.com/Takch02>

BLOG : <https://velog.io/@takch02/posts>

EMAIL : lim70kr38@gmail.com